P3834R0
Defaulting the Compound Assignment Operators

Published Proposal, 2025-07-09

Authors:

Matthew Taylor

Alex (Waffl3x)

Oliver Rosten

Audience:

SG17

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The compound assignment operators have a universally-accepted default meaning. This paper proposes reducing boilerplate by being able to default those operators.

Table of Contents

§ 1. Introduction

The default meaning of the compound assignment operators is universally known and only rarely overloaded to mean anything different. The behaviour of `operator+` and `operator+=` with respect to each other is something which does not change in user code. As such, we propose that the language codify this sensible default and allow such operator overloads to be defaulted. The result, in each case, is an implicitly created function body with the generally accepted meaning. This would cut down on unnecessary boilerplate, reduce the surface area for user mistakes, and make operators which have an exotic implementation stand out in user code by virtue of not being defaulted.

The status quo of the compound assignment operators comes with a lot of boilerplate, which makes for longer code which is slightly harder to reason about. Consider the most basic kind of arithmetic type - a class which wraps an integer:

```
struct int_wrapper{
    int data_;

    int_wrapper(int in) : data_{in} {}

};

int_wrapper operator+(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ + rhs.data_};
}
int_wrapper& operator+=(int_wrapper& lhs, int_wrapper rhs){
    return lhs = lhs + rhs;
}
int_wrapper operator-(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ - rhs.data_};
}
int_wrapper& operator-=(int_wrapper& lhs, int_wrapper rhs){
    return lhs = lhs - rhs;
}
int_wrapper operator*(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ * rhs.data_};
}
int_wrapper& operator*=(int_wrapper& lhs, int_wrapper rhs){
    return lhs = lhs * rhs;
}
int_wrapper operator/(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ / rhs.data_};
}
int_wrapper& operator/=(int_wrapper& lhs, int_wrapper rhs){
    return lhs = lhs / rhs;
}
int_wrapper operator%(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ % rhs.data_};
}
int_wrapper& operator%=(int_wrapper& lhs, int_wrapper rhs){
    return lhs = lhs % rhs;
}
```

None of the individual operations on this class are complex, nor is what the overall class models complex; however the amount of boilerplate required to represent this simple idea means that we spend 50 lines of code covering the basic supported operations (more if the class also emulates bitwise operations). Users of C++ frequently cite complexity and boilerplate as major pain points, sometimes even as a higher frustration than safety [CppDev-2025]. We take this as motivation to simplify things by reducing user boilerplate by defaulting the compound assignment operators. After this change, the above code could be rewritten as:

```
struct int_wrapper{
    int data_;

    int_wrapper(int in) : data_{in} {}

};
int_wrapper operator+(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ + rhs.data_};
}
int_wrapper operator-(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ - rhs.data_};
}
int_wrapper operator*(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ * rhs.data_};
}
int_wrapper operator/(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ / rhs.data_};
}
int_wrapper operator%(int_wrapper lhs, int_wrapper rhs) {
    return int_wrapper{lhs.data_ % rhs.data_};
}

int_wrapper& operator+=(int_wrapper lhs, int_wrapper rhs) = default;
int_wrapper& operator-=(int_wrapper lhs, int_wrapper rhs) = default;
int_wrapper& operator*=(int_wrapper lhs, int_wrapper rhs) = default;
int_wrapper& operator/=(int_wrapper lhs, int_wrapper rhs) = default;
int_wrapper& operator%=(int_wrapper lhs, int_wrapper rhs) = default;
```

We consider this a significant improvement - it is easier for the user to parse this code and see that the class supports the basic arithmetic operations, and the fact that the compound assignment operations are supported is clear and their semantics are easily understood.

We would be interested in applying this change to the library specification, in order to remove boilerplate function definitions while maintaining the same semantics. The authors intend to bring a paper to do this pending interest in the language feature.

> NOTE: This proposal covers the scope of all overloadable operators which have a compound assignment form, namely `+`, `-`, `*`, `/`, `%`, `^`, `&`, `|`, `<<`, and `>>`. For the sake of simplicity, most examples will be written in terms of `operator+` and `operator+=`, but all operators follow the same semantics.

§ 2. Design Principles

It is worth noting the principles which we use to guide our design. The overall goal is a net reduction in language complexity and unnecessary boilerplate, however we want to list specifically how we intend to keep to this goal.

1. **The defaulted semantics of an operator must be universally known to be its default behaviour:** Otherwise classes would be significantly harder to reason about, not easier.
2. **A defaulted operator must not be harder to reason about than its canonical implementation:** Otherwise that would obviate the purpose of the proposal. This imposes a low complexity ceiling on how these operations can be specified and prevents certain "clever tricks" or conditional behaviours which might appear to help at first glance.
3. **A class' supported operations should be clearly visible in code and easy to reason about:** This has been previously reinforced by EWG feedback [P1046-EWG] and usually means that a class' supported operations should be reasonably local to the class definition.
4. **Replacing the implementation of a defaultable operator with `= default;` must not produce surprising results:** In most cases, this means that the behaviour will either be identical to before the change, or produce an error. Users should not be surprised if an overload which previously exhibited non-defaulted behaviour changes its semantics when defaulted.

§ 3. Proposal

> NOTE: In this section we use the non-standard term "underlying operator" to refer to the binary arithmetic or bitwise operator which a compound assignment operator calls. For example, the underlying operator of `operator+=` is `operator+`.

We propose that each compound assignment operator overload be a defaultable function. We permit the following signatures on compound assignment operations operating on types `A` and `B` (where `A` and `B` may be the same type):

```
A& operator+=(A& lhs, const B& rhs) = default;
A& operator+=(A& lhs, B rhs)        = default;
A& operator+=(A& lhs, B&& rhs)      = default;
```

The allowance for `rhs` to be passed by both `const` reference and value is consistent with the existing rules for defaulted equality and comparison operators from C++20 ([class.compare.default]).

Since existing wording for defaulting the copy and move assignment operators explicitly permits them to be lvalue or rvalue qualified ([dcl.fct.def.default]), for consistency we follow this model and so also tentatively allow the rather arcane:

```
A& operator+=(A&& lhs, const B& rhs) = default;
A& operator+=(A&& lhs, B rhs)        = default;
A& operator+=(A&& lhs, B&& rhs)      = default;
```

[P2953] seeks to forbid such signatures by requiring that all defaulted assignment operators must operate on an lvalue reference type. For further discussion, see § 6.3 P2953 Forbid defaulting operator=(X&&) &&.

Regardless, for all signatures above the function body will be equivalent to:

```
{
    return lhs = std::move(lhs) + std::move(rhs);
}
```

though for cases where B is captured by `const&`, the second move is redundant and so degenerates to

```
{
    return lhs = std::move(lhs) + rhs;
}
```

Note that we propose to support all ways of expressing such signatures - as traditional member functions, as explicit object functions, or as free functions (declared as friends or directly in the appropriate `namespace`). For example, all of the below operations are supported and behave equivalently:

```
struct S1{
  int value_;
  S1 operator+(S1 rhs) const { /*...*/ }
  S1& operator+=(S1 rhs) = default;
};

struct S2{
  int value_;
  S2 operator+(S2 rhs) const { /*...*/ }
  S2& operator+=(this S2& lhs, S2 rhs) = default;
};

struct S3{
  int value_;
  S3 operator+(S3 rhs) const { /*...*/ }
};
S3& operator+=(S3& lhs, S3 rhs) = default;
```

Also note that we do not require the underlying operator to be a member function of the class, only that it is visible from the context of the function body of the defaulted compound assignment operator.

The surrounding rules for the validity of defaulted compound assignment operators is consistent with those for defaulted functions which are already defined in the language, such as the rules specified in [dcl.fct.def.default], and we shall summarise them here, returning to the example parameter types `A` and `B` used at the start of this section.

If the return type of the defaulted compound assignment operator is not `A&`, or if either `A` or `B` is not a complete type in the context in which the function is defaulted, the program is ill-formed. If `A` is not move-assignable, or if there is no underlying operator with compatible parameter types which is accessible from the context of the body of the defaulted compound operator function, it is defined as deleted. This is consistent with the existing rules for defaulting special member functions.

We require that the specific overload of the underlying operator which a defaulted compound assignment operator will defer to has a return type of `A`, otherwise the function is defined as deleted. This is to ensure that the semantics of the implicitly-generated function body are well-formed and do the right thing. We do not see this as a difficult restriction as it is widely accepted that the default behaviour of `operator+` is to return a new instance of the same type of its leftmost parameter.

§ 3.1. Minor Edge Cases

The behaviour of explicitly defaulted functions is already laid out in the standard, and our design seeks to be as consistent with the existing behaviour as possible. However, there are a few minor edge cases to consider. Our baseline when designing this feature is to consider the case where `= default;` is replaced with an equivalent function body and to determine whether it results in code which is well-formed and doesn't exhibit surprising behaviour.

§ 3.1.1. cv-qualification and ref-qualification

We forbid a defaulted compound assignment operator from operating on a cv-qualified instance of an object. This is consistent with the existing wording in [class.copy.assign] and [dcl.fct.def.default] which forbids a defaulted copy- or move-assignment operator from being cv-qualified. We see no good reason to deviate from existing practice.

While it is in principle possible to engineer a cv-qualified assignment operator overload which in turn would mean that replacing `= default;` with the canonical function body would produce well-formed code, we do not consider assigning to a `const` object as a sensible default for the language. Similarly, we do not permit the rightmost parameter of a defaulted compound assignment operator to be a `volatile` reference type. This is not supported by defaulted assignment operators and we do not feel it's a sensible enough default for compound assignment operators to be treated differently.

As discussed above, we begrudgingly permit defaulted compound assignment operators to operate on rvalue-ref-qualified types for the sake of consistency. We also note that existing wording for defaulting copy- and move-assignment operators explicitly permits them to be lvalue ref-qualified. As such, the implicit-object non-static member function forms of defaulted compound assignment operator overloads may have any *ref-qualifier*.

§ 3.1.2. Implicit conversion to parameter types of the underlying operator

Consider the following code:

```
struct from_int{
    from_int(int) {} //implicitly constructible from int
};
foo operator+(foo lhs, from_int rhs){ /* ... */}
foo& operator+=(foo& lhs, int rhs) = default;
```

Should this defaulted `operator+=` overload be permitted? If we were to directly replace `= default;` with the canonical function body then the implicit conversion of `rhs` to `from_int` would trigger and the code would work. However, allowing such implicit conversions inside a generated function body also has the potential to increase user confusion by permitting an unexpected overload of the underlying binary operator to be called. We note that the scope of this proposal covers the bit-wise operators, which are frequently overloaded to perform operations other than bit manipulation, and this could lead to potentially surprising conversion paths.

On the other hand, having the above code be ill-formed or the overload defined as deleted because an implicit conversion would happen inside of a function body that doesn't exist in the user's code would be a very surprising and confusing bug. While it is arguable that an implicit conversion bug may be more surprising, we see this as a symptom of user code allowing undesirable implicit conversion paths rather than a defect in the concept of defaulting compound assignment operators. Similarly, making such function calls ill-formed would come with its own complexities; such as users attempting to SFINAE on the validity of such declarations. While it would certainly be possible to engineer user-hostile conversion paths if we permit the above code, we do not consider it likely in reasonable code. We expect that by far the most likely use of this proposal is that users will naturally match the parameter types between their defaulted compound assignment operators and their existing underlying operator overloads as is the common default today.

Considering this, we only require that there exists an underlying operator overload which would be callable if `= default;` were replaced by its canonical function body; not that all parameter types must match exactly.

§ 3.1.3. Explicit object parameter types

We require that an explicit object parameter of a defaulted compound assignment operator must be of the same type of the class in which it is declared, and note that existing wording in [dcl.fct.def.default] already enforces this for other defaulted functions.

§ 3.1.4. Self-assignment

We do not see the need to insert a check for self-assignment in a defaulted compound assignment operation. Our requirement that the underlying operator's return type be by value means that we will always be assigning from a brand new instance of the class, and in the unlikely case that there is potential breakage in either assignment from a different instance or calls to the underlying operator with the same object twice, these must already be handled inside their respective functions.

§ 3.2. `= default` is a function body

We note specifically that `= default;` is a *function-body* which the implementation uses to inject the canonical semantics for that function. As such, declaration-level grammar of explicitly defaulted functions is unaffected. The declaration of an explicitly defaulted function may still be marked `noexcept` or `constexpr`, it may still be constrained with `requires` clauses or have attributes appertaining to it.

Consequently, this paper neither forbids nor requires further grammar changes to permit code such as:

```
class foo{ /*...*/ };
constexpr foo operator+(const foo& lhs, const foo& rhs) noexcept;
constexpr foo& operator+=(foo& lhs, const foo&) noexcept = default;
```

Similarly, there is no magic associated with `= default;` which affects overload resolution. No part of this proposal permits the implicit creation of the operator signatures we are discussing: users must explicitly opt-in. Nor should there be any surprises in overload resolution order. And, while it is not necessary, if the user wishes to explicitly delete their compound assignment operator overloads then no part of this proposal prevents them from doing so.

This also means it is possible for the user to declare multiple defaulted compound assignment operator overloads for a given class, provided that they would otherwise be valid overloads:

```
class bar{ /*...*/ };
bar operator+(const bar& lhs, const bar& rhs);
bar& operator+=(const bar& lhs, const bar& rhs) = default;
bar& operator+=(const bar& lhs, bar&& rhs) = default;

bar operator+(const bar& lhs, int rhs);
bar& operator+=(const bar& lhs, int rhs) = default;
```

§ 4. Are We Creating Tomorrow's Boilerplate?

It is easy to imagine a world where a class contains many defaulted functions and operators. Consider:

```
class foo{
  some_data m_data;

  public:
```

```
  //C++11
  foo() = default;
  foo(bar){ ... }


  //C++20
  auto operator<=>(const foo&) const = default;

  //P3668
  foo& operator++() { ... }
  foo operator++(int) = default;
  foo& operator--() { ... }
  foo operator--(int) = default;

  //This proposal
  foo operator+(const foo&) const { ... }
  foo& operator+=(const foo&) = default;
  foo operator-(const foo&) const { ... }
  foo& operator-=(const foo&) = default;

};
```

It's reasonable to ask whether this reduction in boilerplate itself creates boilerplate which would need to be reduced in future. We consider it preferable for the operations on a class to be explicitly visible in code, and EWG has previously found consensus in agreement ([P1046-EWG]). We argue that reducing it further would violate our second design principle by requiring the user to deduce supported operations rather than simply read them. Ultimately, we don't see this boilerplate as a symptom of a problem with the proposal but just a consequence of the fact that C++ requires a certain level of verbosity when describing a class' supported features. This is not unique to C++ - we also note that an equivalent class in Python, using its dunder methods for supported operations, would have a similar line count.

Further discussion on issues with implicit generation of operators can be found in § 5.6 Rewrite Rule and Implicit Generation of Operators.

§ 5. Alternatives Considered

§ 5.1. Defining `operator+` in terms of `operator+=`

There has historically been some discussion in C++ about whether it is better to implement the compound assignment operator in terms of its underlying operator:

```
struct foo{int x, y;};
foo operator+(const foo& lhs, const foo& rhs){
    return foo{lhs.x + rhs.x, lhs.y + rhs.y};
}
foo& operator+=(foo& lhs, const foo& rhs){
    return lhs = lhs + rhs;
}
```

or to do the reverse:

```
struct foo{int x, y;};
foo& operator+=(foo& lhs, const foo& rhs){
    lhs.x += rhs.x;
    rhs.y += rhs.y
    return lhs;
}
foo operator+(foo lhs, const foo& rhs){
    return lhs += rhs;
}
```

As such, it is reasonable to ask whether we should permit defaulting the underlying operator instead of, or in addition to, the compound assignment operator.

We argue that this would be a misstep. From first principles it makes far more sense to default the composition of `+` and `=` than it does to require the user define the composite operation in order to generate the behaviour of one of the elementary operations. Defaulting the underlying operator comes with additional potential for confusion, for example a user might have trouble intuiting that `foo operator+(const foo&, const foo&) = default;` is a function which implicitly calls `operator+=` to perform addition as opposed to some other purpose (e.g. memberwise addition for aggregates).

The possibility of defaulting both operator categories also comes with many issues. First and foremost, checking for the validity of a defaulted operation becomes significantly harder if either the compound

assignment operator and the underlying operator may be defaulted. Similarly, it opens up a difficult question on what to do if both are defaulted. It occupies twice as much syntactic space and is harder to reason about than simply defaulting the compound operators. It opens the door to lengthy discussions about which is the better operator to default and how that may differ from handwritten operators.

David Stone explores this same question in [P1046], and writes:

> One interesting thing of note here is that the operator+= implementation does not know anything about its types other than that they are addable and assignable. Compare that with the operator+ implementations from the first piece of code where they need to know about sizes, reserving, and insertion because operator+= alone is not powerful enough. There are several more counterexamples that argue against operator+= being our basis operation, but most come down to the same fact: operator+ does not treat either argument -- lhs or rhs -- specially, but operator+= does.
> The first counterexample is std::string (again). We can perform `string += string_view`, but not `string_view += string`. If we try to define operator+ in terms of operator+=, that would imply that we would be able to write `string + string_view`, but not `string_view + string`, which seems like an odd restriction. The class designer would have to manually write out both += and +, defeating our entire purpose.
>
> The second counterexample is the `bounded::integer` library and `std::chrono::duration`. `bounded::integer` allows users to specify compile-time bounds on an integer type, and the result of arithmetic operations reflects the new bounds based on that operation. For instance, `bounded::integer<0, 10> + bounded::integer<1, 9> => bounded::integer<1, 19>`. `duration<LR, Period> + duration<RR, Period> => duration<common_type_t<LR, RR>, Period>` If operator+= is the basis, it requires that the return type of the operation is the same as the type of the left-hand side of that operation. This is fundamentally another instance of the same problem with `string_view + string`.
>
> The third counterexample is all types that are not assignable (for instance, if they contain a const data member). This restriction makes sense conceptually. An implementation of operator+ requires only that the type is addable in some way. Thanks to guaranteed copy elision, we can implement it on most types in such a way that we do not even require a move constructor. An implementation of operator+= requires that the type is both addable and assignable. We should make the operations with the broadest applicability our basis.

Overall, we argue that the only sensible option to default are the compound assignment operators, and while there have historically been some benefits to implementing `operator+` in terms of `operator+=` they do not outweigh the issues which defaulting the underlying operator in terms of its compound assignment operator would bring.

§ 5.2. A Perfect Forwarding Compound Assignment Operator Template

Given that our proposal requires users to place two declarations of a compound operator in their class if they want to support the rightmost argument being copied or moved as appropriate into the underlying operator function, it may be tempting to propose supporting the ability to default an operator overload function template, which forwards the argument. For example:

```
A& operator+=(A& lhs, std::convertible_to<B> auto&& rhs) = default;
```

While we agree this looks good at first pass, it opens the door to a lot of issues in terms of semantics and consistency with the rest of the language. Nowhere else in the language are explicitly defaulted functions permitted to be function templates, so permitting them here would be an inconsistency. Equally, it raises a question of whether we should only permit the above signature or whether any valid function template signature should be defaultable. We argue that defaulting function templates is a separate proposal with its own open design questions to consider and should not be considered as a requirement for this proposal. Similarly, we are not entirely persuaded that a perfect forwarding template is used as a common sensible default in real code.

§ 5.3. A Magic Pseudo Operator

One idea which has occasionally been discussed is to introduce some "magic" pseudo operator which when defaulted in a class implicitly adds all generatable operators to it, for example:

```
class foo{
    foo operator+(const foo& rhs) const { /*...*/ }
    foo& operator-=(const foo& rhs) { /*...*/ }

    auto operator@=() = default; //Implicitly adds operator+= and operator-
};
```

We think this is a little too cute, and do not prefer it to explicitly defaulting the compound operators. Defining a construct which looks like an operator but is a handle on some internal function injection could be very confusing for users. There are difficult questions about how the function semantics of `operator@=` should be specified - for example what return type it should claim to have, what parameters it should accept, and whether it is considered to be acting on `const` or mutable instances of the class. Equally it could not be constrained to being an apparent nonstatic member function - there are good reasons users might have to define their operator overloads as free functions and

`operator@=` would have to be a free function itself in order to be able to see them and generate code accordingly. This raises additional points of confusion if some operators are defined in a different TU from `operator@=` and so cannot have their counterparts generated, and whether multiple "definitions" of `operator@=` should be permitted.

What such a pseudo operator attempts to achieve is to give the user some declaration they can insert into a class to say "give me the default operations"; but C++ already has such a declaration with `= default` and we are not persuaded that an additional one would be an improvement. Overall we do not consider this a workable idea for the language.

§ 5.4. Reflection Solutions

When generative reflection arrives, it is conceivable that it would be possible to write some `add_generatable_operators()` metafunction which reads the valid operations for some class and generates the "default" behaviour of any others based on them. We agree, and do not see this proposal as competing with reflection in that space. Indeed, the ability to simply inject `= default;` as the function body could make such a metafunction far easier to write.

However, we also hold that such a metafunction would be a poor candidate for standardisation. There are many possibly generatable operators following many different possible paths - not just through operators which can be defaulted but also through compositions of other operators, such as obtaining subtraction through addition and unary negation, or other compositions such as those explored in [P1046]. It is not clear what the scope of a "standard" metafunction to do this should be; nor is it clear whether it should be expanded if new operators or operator generation methods are added in future standards.

It is also conceivable that metafunctions can be written to add each generatable operator in turn, or each family of operators. We see this as a standardisation issue in itself - a monolith of standard metafunctions for specific cases is not desirable, and it may not be easy to agree on which families of operators should be grouped together. This aside, however, we do not find that these solutions are as easily readable as simply defaulting the operators. The code to generate these is complex, and the mechanism to inject them may not be immediately clear to non-expert C++ developers. Consider this example from Matthias Wippich, following the design laid out in [P3294]:

```
#include <experimental/meta>

consteval void generate_arithmetic(std::meta::info R) {
  // note that is_incomplete_type got replaced with is_complete_type
  auto befriend = is_incomplete_type(R) ? ^^{friend} : ^^{};
```

```
  auto inject = [&](std::meta::info op, std::meta::info rewritten) {
    auto tokens = ^^{
      template <typename U>
      \tokens(befriend) typename[:\(R):]& \tokens(rewritten)(typename[:\(R
        requires requires {
          { lhs \tokens(op) std::forward<U>(rhs) };
        }
      {
        lhs = std::move(lhs) \tokens(op) std::forward<U>(rhs);
        return lhs;
      }
    };
    queue_injection(tokens);
  };

  inject(^^{+}, ^^{operator+=});
  inject(^^{-}, ^^{operator-=});
}



// USAGE
struct Vec2 {
  float x, y;
  Vec2 operator+(Vec2 const& other) const { return {x + other.x, y + other
  Vec2 operator-(Vec2 const& other) const { return {x - other.x, y - other

  consteval { generate_arithmetic(^^Vec2); }
};

int main() {
  Vec2 a{1, 2}, b{3, 4};
  a += b;
  a -= b;
}
```

Godbolt here

This code is a complex way of achieving the same overall effect as defaulting these operators. Additionally, the call to `generate_arithmetic` is a different shape from common declarations; and

may not be easily coupled with the class definition. We are not convinced that the above code should be a common pattern to solve everyday code problems - it is far easier to understand the operations which a class supports by reading function declarations than it is to parse each `consteval` block and mentally unwrap an opaque function call to some reflection function. A user may also declare their underlying operators as free functions, which means that the `consteval` block may exist outside of the class definition in order to be able to see the supported operations. This in turn makes the injection function more error-prone and we argue that injection functions which are so decoupled from the definitions of the classes on which they operate are far harder to reason about than a defaulted function declaration.

We hold that while such a reflection based solution may be of use for specific situations and specific users, it does not mean that we cannot recognise the canonical default behaviour of these functions by defaulting them as a more general-purpose solution.

§ 5.5. Free Function Compound Assignment Operator Templates

It would be possible to define some global operator templates to support compound assignment, which are made opt-in by some template bool `enable_compound_addition` or some `[[=generate_compounds]]` annotation. This has similar problems to other reflection-based solutions - the granularity of the opt-in switch is difficult to determine and the operations which a class supports become harder to reason about as the user must go hunting for the special tag which describes functionality rather than traditional function declarations. Such a solution comes with additional problems - free function templates may not be generated if the user intends for an implicit conversion to occur.

Consider the classic example where implicit conversions fail for the left argument of a binary operator:

```
template<typename T>
class wrap{
    T t_;

    public:
    wrap(const T& value) : t_{value} {} //Implicitly constructible from T

    auto& get(this auto&& self){
        return self.t_;
    }
};
```

```
template<typename T>
wrap<T> operator+(const wrap<T>& lhs, const wrap<T>& rhs){
    return wrap<T>{lhs.get() + rhs.get()};
}

int main(){
    wrap<int> wr{0};
    0 + wr; //Ill-formed: No match for operator+
}
```

The traditional solution to this problem is to declare `operator+` as a hidden friend, but this would not be possible to do for the default compound operators if they were free functions in some header.

We argue that putting function templates for supported operations into standard library headers makes code harder to reason about, not easier. These definitions would not be local to the class in code, would require memorising the special rule that X tag means Y behaviour, and too closely resembles `std::rel_ops`. The idea was dropped.

§ 5.6. Rewrite Rule and Implicit Generation of Operators

At first glance, it may seem tempting to permit a spacehip operator-style rewrite rule of `a += b` to `a = a + b`. However this comes with many potential traps and sources of confusion. We maintain that it is preferable for a class' supported operations to be visible in code, and that implicit generation of supported operations is harder to reason about than the status quo in this case. There are additional concerns - a rewrite rule following current C++ convention would be forced to evaluate `a` twice when rewritten rather than just once. If `a` were the result of some function call `f()`, then the function would be called a second time implicitly, which would at the very least be surprising. We can only see two possibilities to avoid this while maintaining rewrite. The first is a carveout in the wording to make such rewrites "magic" in only evaluating the left operand once, and we do not believe that the complexity of this special case justifies its benefits. The second is to have the call to `a += b` crystalise some implicit actual function or function object such that `a` is only evaluated once as it is bound to a parameter or variable. We do not think that this is a viable path either, as introducing a second mechanism by which rewrite rules work makes an already complex topic significantly harder to reason about and remember. It introduces ODR issues in and of itself if such an operator is generated in a header; but also if the user attempts to declare a traditional operator overload after the implicit one has been generated.

Such a change would come with a specification burden to other papers. Consider the `declcall` operator as proposed by [P2825]. Should `declcall(a += b)` be well-formed if used in a translation unit

where the compiler has not generated an implicit function to perform the operation? The call to `a += b` at that place in code would itself be well-formed and call some kind of function, but the dedicated tool to reach that function may have to be conditionally ill-formed depending on what code already exists in the current translation unit. Should such an implicitly generated function be a reflectable member of whatever namespace it is considered to be declared in? Must users explicitly generate these functions with a series of `a += b`, `a -= b`, `a *= b`, and so on before they are reflectable? Or should we be able to reflect on all possible *potential* functions which would be possible to generate and use in user code at that point in that TU? A reflection of something which does not exist in source code seems quesitonable, but so does the inability to reflect on all of a class' supported operations.

An alternative path we could take would be to follow the approach of the special member functions with operations which are implicitly present even if not user-declared. For example, we could propose that following the declaration of an underlying operator, that there implicitly is a compound assignment operator present in the same way that a class' move-assignment operator is implicitly present. This would solve some of the above issues - user level declarations of a given compound assignment operator overload would not conflict with the implicit one - they would instead be considered to work similarly to a user-provided special member function and subsume it. But it comes with many issues of its own. The special member functions are required to be member functions of their classes for good reason, whereas compound assignment operations may be free functions and as such may exist anywhere in code. We do not think it is reasonable for compilers or for users to reason about potentially-non-default implicit operator declarations which may be present anywhere in the current TU. The existing implicit presence of special member functions has historically been a cause of confusion and difficulty for beginners; and we don't see this idea as less confusing. Equally, this design shares issues with rewrite rules and implicit generation of operation overloads in general - it is opt-out, rather than opt-in. Existing code which made the decision to not support compound assignment operations must be updated with explicitly deleted function overloads in order to maintain this decision, otherwise code which is ill-formed today would become code which is well-formed but does the wrong thing.

Overall we are not convinced that a rewrite rule is the best design, nor is it practically feasible, so the idea was dropped.

§ 6. Other Papers in this Space

We are not aware of any other active papers which seek to automatically generate compound assignment operator overloads, however there are some papers which occupy the more general space of explicitly defaulted functions, and we note them here:

§ 6.1. P1046 Automatically Generate More Operators

[P1046] sought to add additional rewrite rules to the language, following the pattern of the spaceship operator in C++20, and this included rewriting the compound assignment operators into some composition of the underlying operator and assignment operator. P1046 is currently parked in favor of a more fine-grained proposal [P3039], which does not overlap with this paper.

§ 6.2. P2952 auto& operator=(X&&) = default;

[P2952] argues that explicitly defaulted functions should be permitted to use placeholder types in their return types, so long as the chosen placeholder type would deduce to the correct return type if the return statement in the canonical function body were present. For example, `operator==` is permitted to use `auto` but not `auto&` or `auto*` as its declared return type.

This paper is compatible with P2952, and as defaulted compound assignment operators much always have a return type of `A&`, they follow the same placeholder return type pattern as defaulted copy- and move-assignment operators:

EXAMPLE 1 ¶

The behaviour of placeholder return types for a defaulted `operator+=(A&, const B&)`:

```
struct C{
    auto& operator+=(A&, const B&) = default;              //Well-formed, d
    decltype(auto) operator+=(A&, const B&) = default;     //Well-formed, d
    auto&& operator+=(A&, const B&) = default;             //Well-formed, d
    auto* operator+=(A&, const B&) = default;              //Ill-formed, de
    auto operator+=(A&, const B&) = default;               //Ill-formed, A
    const auto& operator+=(A&, const B&) = default;        //Ill-formed, co
};
```

As P2952 is in the CWG queue for C++29, we will include wording relative to it in this paper.

§ 6.3. P2953 Forbid defaulting operator=(X&&) &&

[P2953] seeks to explicitly ban defaulted copy- or move-assignment operators from operating on an rvalue ref-qualified objects; either by defining them as deleted or by making them ill-formed depending on WG21 feedback. As compound assignment operators are still assignment operators, and signatures such as `A& operator+=(A&& lhs, const B& rhs) = default;` seem generally implausible, we recommend that defaulted compound assignment operators fall under the same scope as defaulted copy- and move-assignment operators with respect to P2953 and should also be defined as deleted or made ill-formed as that paper progresses.

We have reached out to the author of P2953 and he agrees with this assessment. Consequently, as P2953 progresses through the WG21 process, we will revise our design and wording in parallel to it to maintain consistency.

§ 6.4. P3668 Defaulting Postfix Increment and Decrement Operations

[P3668] proposes that postfix increment and decrement operators should be defaultable, in the same way that this paper proposes that compound assignment operators should be defaultable. We see no conflicts between the two papers, and note that the rules surrounding explicitly defaulted functions are consistent between the two papers, and consistent with the existing standard.

As P3668 is currently in CWG for C++29, we will provide wording relative to it.

§ 7. Acknowledgements

Many thanks to Matthias Wippich for his insight into reflection, and for providing the example in § 5.4 Reflection Solutions. Many thanks also go to Rodrigo Fernandes for reviewing the initial draft.

§ References

§ Informative References

[CppDev-2025]
2025 Annual C++ Developer Survey 'Lite'. URL: https://isocpp.org/files/papers/CppDevSurvey-2025-summary.pdf

[P1046]

David Stone. *Automatically Generate More Operators*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1046r2.html

[P1046-EWG]

EWG. *EWG comments from the Belfast meeting*. URL: https://wiki.edg.com/bin/view/Wg21-belfast/P1046-EWG

[P2825]

Gašper Ažman, Bronek Kozicki. *Overload resolution hook: declcall(unevaluated-call-expression)*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2825r5.html

[P2952]

Arthur O'Dwyer, Matthew Taylor. *auto& operator=(X&&) = default*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2952r2.html

[P2953]

Arthur O'Dwyer. *Forbid defaulting operator=(X&&) &&*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2953r1.html

[P3039]

David Stone. *Automatically Generate operator->*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p3039r0.html

[P3294]

Andrei Alexandrescu; Barry Revzin; Daveed Vandevoorde. *Code Injection with Token Sequences*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3294r2.html

[P3668]

Matthew Taylor, Alex. *Defaulting postfix increment and decrement operations*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3668r2.html